LILYTORCH: A GPU-Accelerated Immersed-Boundary Navier–Stokes Solver for Closed-Loop Neuromechanical Simulation of Aquatic Locomotion

Andrea Ferrario

***Abstract*—Simulating freely-swimming robots and animals requires coupling a multibody dynamics engine with a fluid solver that is fast enough for closed-loop neuromechanical experiments yet accurate enough to capture viscous boundary layers and unsteady wakes. We present LILYTORCH, an open-source, GPU-accelerated incompressible Navier–Stokes solver that implements a second-order Boundary Data Immersion Method (BDIM2) on a staggered Cartesian grid and couples two-way with the MUJOCO physics engine through the FARMS framework. The solver supports both analytical signed-distance functions (spheres, capsules, boxes, fish spines) and mesh-based bodies (STL/OBJ through OPEN3D and fast-marching), handles per-link articulated kinematics, and offers a choice of spectral (DCT / free-space Green's function) or variable-coefficient multigrid pressure-projection solvers. Optional Smagorinsky large-eddy closure, Carreau / Herschel–Bulkley non-Newtonian rheology, and a sponge damping layer extend the solver to a range of bio- and soft-robotic scenarios. Because the complete pipeline (advection–diffusion, projection, BDIM, force integration) is expressed in PYTORCH tensors, every hot kernel is JIT-compiled via `torch.compile` and runs on a single GPU without domain decomposition. We describe the continuous and discrete formulations, detail how rigid, articulated, and mesh bodies are integrated with MUJOCO, and discuss the implications of the proposed coupling for embodied studies of aquatic locomotion.**

***Index Terms*—Biologically-inspired robots, Computational fluid dynamics, Fluid–structure interaction, Immersed boundary method, Simulation, Swimming robots.**

I. INTRODUCTION

UNDERSTANDING how neural control, body mechanics, and the surrounding fluid interact during swimming is a long-standing question in both neuroscience and robotics. Anguilliform swimmers [1], zebrafish larvae [2], [3], salamanders walking and swimming [4], and an increasing number of soft and articulated aquatic robots [5], [6] all exhibit behaviour that can only be explained by closing the loop between the controller, the body, and the hydrodynamic environment [7]. Building computational platforms that *embody* this loop — i.e. that simulate controller, multibody dynamics, and fluid together in real- or near-real-time — remains a key methodological challenge.

Manuscript submitted to IEEE Robotics and Automation Letters. The author is with the BioRobotics Laboratory, EPFL, Lausanne, Switzerland. E-mail: andrea.ferrario@epfl.ch.

Classical body-conforming Navier–Stokes solvers based on finite volumes or spectral elements (e.g. OPENFOAM [8], NEK5000 [9]) deliver excellent fidelity but require mesh motion or remeshing when the geometry is articulated or freely moving; they are also too slow to be driven interactively from a robotics engine. Immersed-boundary (IB) methods [10], [11] circumvent these issues by solving the fluid on a fixed Cartesian grid and representing the body as a forcing or blending term. Among IB variants, the *Boundary Data Immersion Method* (BDIM) [12], [13] blends the fluid velocity with the prescribed body velocity through a smoothed Heaviside function derived from a signed-distance field, and has been shown to achieve second-order accuracy on a wide range of bio-inspired flows [14], [15]. The WATERLILY.JL package [16] demonstrated that BDIM can be implemented concisely on top of modern array frameworks; our work adopts the same philosophy in PYTORCH and extends it to two-way coupling with a general-purpose multibody engine.

On the robotics side, MUJOCO [17] has become the reference engine for articulated rigid-body dynamics and is widely used for legged and swimming locomotion. The FARMS framework [18] provides a high-level neuromechanical abstraction on top of MUJOCO, with central-pattern generators [19], sensor buses, and extension hooks. However, neither MUJOCO nor FARMS natively models a viscous fluid: aquatic experiments have historically relied on drag coefficients or added-mass approximations, which cannot reproduce wake reattachment, vortex shedding, or inter-body interference. Plugging a fully-resolved CFD solver into this stack requires: (i) a fluid formulation that accepts arbitrarily-moving, articulated geometry at every time step; (ii) a body representation that covers analytical primitives as well as arbitrary triangular meshes (for which MUJOCO uses STL/OBJ); and (iii) a pressure-projection path that tolerates per-step changes in body mask, density, and viscosity.

This paper describes LILYTORCH, an open-source PYTORCH-based [20] incompressible Navier–Stokes solver that we designed to fill this gap. Its contributions are:

- **A single-GPU, two-way coupled CFD+MUJOCO pipeline.** Every time step the fluid solver reads link poses from MUJOCO, recomputes per-link signed-distance functions (SDFs), performs a BDIM2 Heun predictor–

corrector step, integrates hydrodynamic forces, and writes them back as external wrenches. The entire pipeline runs on one GPU without inter-process communication.

- **Unified analytical / mesh body handling.** A common `Body` abstraction exposes spheres, capsules, boxes, NACA fish-spine profiles, and STL/OBJ meshes (loaded via OPEN3D [21] with fast-marching sign propagation [22]) through the same BDIM interface. Articulated animats are supported through composite bodies that take the pointwise union of per-link SDFs.
- **Choice of pressure solvers.** A spectral DCT solver (Neumann boundaries), a free-space Green's function convolution [23] for unbounded domains, and a geometric multigrid [24] preconditioned CG (MGCG) solver for variable-coefficient problems arising from the BDIM mask or variable density.
- **Advection, turbulence, and rheology models.** Six advection schemes (QUICK [25], ADBQUICKEST, CUBISTA [26], van Leer [27], CDS, and semi-Lagrangian backtracing [28]); an optional Smagorinsky [29] large-eddy closure; Carreau and Herschel–Bulkley non-Newtonian models [30]; and a quadratic sponge layer for open-domain emulation.
- **GPU kernel fusion.** All hot paths (advection–diffusion, multigrid smoother and V-cycle, force integration, SDF evaluation) are expressed as pure functions that can be JIT-fused through `torch.compile`, taking advantage of CUDA-graph capture.

The remainder of this paper is organised as follows. Section II presents the continuous equations, the discrete numerical schemes, the BDIM2 immersed-boundary formulation, the analytical and mesh body classes, and the coupling with MUJOCO through FARMS. Section III discusses design trade-offs, comparisons with existing tools, limitations, and planned extensions. Experimental validation and quantitative results are deferred to a companion section.

II. METHODS

A. Governing equations

LILYTORCH solves the incompressible Navier–Stokes equations in primitive variables,

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u} = -\frac{1}{\rho}\nabla p + \nu\,\nabla^2\mathbf{u}, \tag{1}$$

$$\nabla\cdot\mathbf{u} = 0, \tag{2}$$

where $\mathbf{u} = (u, v)$ in 2-D or (u, v, w) in 3-D, p is the pressure, ρ the (constant) fluid density, and ν the kinematic viscosity. The incompressibility constraint (2) is enforced by the classical Chorin–Témam fractional-step method [31], [32]: an explicit provisional velocity $\tilde{\mathbf{u}}$ is first advanced with the momentum terms, and a Poisson problem is then solved to obtain the pressure correction.

B. Staggered MAC grid and time integration

The domain $\Omega \subset \mathbb{R}^d$ ($d \in \{2, 3\}$) is discretised on a Marker-and-Cell (MAC) staggered grid [33]: pressure is stored at cell centres and each velocity component on its own face grid (u at x-faces, v at y-faces, w at z-faces). One ghost cell is added on each side of Ω, giving array shapes $(N_x{+}2){\times}(N_y{+}2)\,[{\times}(N_z{+}2)]$; boundary conditions are enforced through image/mirror ghost values for Dirichlet or zero-gradient ghost values for Neumann faces.

Time integration follows Heun's method (explicit RK2) as in WATERLILY.JL [16]:

1) **Predictor:** advection–diffusion of $\mathbf{u}^n$ gives $\mathbf{u}^*$; BDIM blends $\mathbf{u}^*$ with the body velocity (Section II-C); pressure projection with weight $w{=}1$ produces $\tilde{\mathbf{u}}$.
2) **Corrector:** advection–diffusion of $\tilde{\mathbf{u}}$ gives $\mathbf{u}^{**}$; a half-step rebase $\mathbf{u}^{**} \leftarrow \frac{1}{2}(\mathbf{u}^{**} + \mathbf{u}^n)$ is applied; BDIM is re-blended; a second pressure projection with weight $w{=}\frac{1}{2}$ returns $\mathbf{u}^{n+1}$.

The advection–diffusion sub-step itself uses forward Euler. CFL stability requires $\Delta t \le \min(h)\,/\,(v_{\max} + 3(\nu + \nu_t^{\max}))$, where ν_t is the (optional) Smagorinsky eddy viscosity.

a) Advection schemes: Six choices are exposed for the flux $\nabla\cdot(\mathbf{u}\otimes\phi)$: third-order upwind QUICK [25] with median limiter; a TVD-limited variant (ADBQUICKEST); the high-resolution CUBISTA [26]; van Leer's limiter [27]; plain central differences; and Stam's unconditionally stable semi-Lagrangian back-tracing [28] for very large Δt at the cost of numerical diffusion.

b) Optional Smagorinsky LES: When the grid is too coarse to resolve all eddies, the subgrid-scale closure of Smagorinsky [29] adds an eddy viscosity $\nu_t = (C_s\Delta)^2|\bar{S}|$, with $|\bar{S}| = \sqrt{2\,\bar{S}_{ij}\bar{S}_{ij}}$ the resolved strain-rate magnitude and $\Delta = h$. The momentum diffusion becomes the variable-coefficient operator $\nabla\cdot\big[(\nu+\nu_t)\nabla\mathbf{u}\big]$, discretised by arithmetic averaging of $(\nu{+}\nu_t)$ at the MAC faces. Setting $C_s{=}0$ recovers the constant-viscosity path at zero overhead.

C. Boundary Data Immersion Method

Bodies are represented by a signed-distance function (SDF) $d(\mathbf{x})$, with $d > 0$ in the fluid, $d = 0$ on the surface, and $d < 0$ inside the body. Two smooth kernel functions of half-width $\varepsilon{=}1.5h$ are built from d: a Heaviside-like indicator

$$\mu_0(d) = \begin{cases} 0, & d \le -\varepsilon, \\ \frac{1}{2}\Big[1 + \frac{d}{\varepsilon} + \frac{\sin(\pi d/\varepsilon)}{\pi}\Big], & |d| < \varepsilon, \\ 1, & d \ge \varepsilon, \end{cases} \tag{3}$$

and a first-moment kernel $\mu_1(d)$ obtained by integrating μ_0. At each RK2 sub-step the provisional velocity ϕ is blended with the prescribed body velocity $\mathbf{v}_b$ via the BDIM2 *meta-equation* [12], [13]:

$$\phi_{\text{out}} = \mu_0\,(\phi - \mathbf{v}_b) + \mathbf{v}_b + \mu_1\,\hat{\mathbf{n}}\cdot\nabla(\phi - \mathbf{v}_b), \tag{4}$$

with $\hat{\mathbf{n}} = \nabla d/|\nabla d|$. Inside the body $\mu_0, \mu_1 \to 0$ and $\phi_{\text{out}} = \mathbf{v}_b$; in the fluid $\mu_0 \to 1, \mu_1 \to 0$ and ϕ is recovered; the transition layer provides second-order enforcement of the no-slip boundary condition without sub-grid reconstruction.

The pressure-projection step is likewise modified to respect the body mask,

$$\nabla\cdot\left(\frac{w\,\Delta t}{\rho}\,\mu_0\,\nabla p\right) = \nabla\cdot\tilde{\mathbf{u}}, \tag{5}$$

so that the pressure gradient vanishes inside the body and the projection remains well-posed at the surface.

a) Force integration: Hydrodynamic forces on a body are recovered by integrating the Cauchy stress tensor over a smoothed surface using an ε- width mollified Dirac:

$$\mathbf{F}_{\text{visc}} = \int (\boldsymbol{\sigma}\cdot\hat{\mathbf{n}})\,\delta_\varepsilon(d-\varepsilon)\,\mathrm{d}V, \tag{6}$$

$$\mathbf{F}_{\text{pres}} = -\int p\,\hat{\mathbf{n}}\,\delta_\varepsilon(d)\,\mathrm{d}V, \tag{7}$$

with $\delta_\varepsilon(d) = [1+\cos(\pi d/\varepsilon)]/(2\varepsilon)$. Torques are accumulated directly from component sums to avoid allocating full moment-arm tensors on the GPU.

D. Pressure-projection solvers

LILYTORCH provides three Poisson back-ends.

a) FFT / DCT (Neumann): When every face carries $\partial p/\partial n{=}0$, the discrete Neumann Laplacian is diagonalised by the type-II Discrete Cosine Transform [34], $\lambda_k = (2/h^2)[\cos(\pi k/N)-1]$, and the solve reduces to DCT–divide–iDCT. This is the fastest path and is used for the majority of validation benchmarks.

b) Free-space Green's function: For unbounded domains the RHS is zero-padded to $2N$ and convolved with a regularised Green's function. In 2-D we use the eighth-order algebraic smoothing of Hejlesen et al. [23]; in 3-D the Gaussian/erf regularisation $G(r) = -(4\pi r)^{-1}\mathrm{erf}(r/\sqrt{2}\sigma)$ with $\sigma{=}h$. Green's functions are precomputed once and cached on disk.

c) Geometric multigrid / MGCG: When the BDIM mask or a variable fluid/body density makes (5) genuinely variable-coefficient, we solve it with a V-cycle multigrid [24] wrapped, if requested, inside a conjugate-gradient iteration (MGCG). The discrete stencil along dimension d reads

$$\frac{c_{d+}p_{i+1} - (c_{d+}+c_{d-})p_i + c_{d-}p_{i-1}}{h^2},$$

with face coefficients $c_{d\pm}$ taken as the harmonic average of μ_0 (or $\Delta t/\rho$ in the variable-density case) at cell faces. Available smoothers are weighted Jacobi ($\omega{=}0.7$ default) and red/black Gauss–Seidel; full-weighting restriction and bilinear (2-D) or trilinear (3-D) prolongation are used. The coarsest level is a trivial $2\times 2\,(\times 2)$ direct solve.

d) Variable-density extension for MUJOCO *coupling:* When the body density ρ_b differs from the fluid ρ_f we assemble an effective density $\rho(\mathbf{x}) = \rho_b + (\rho_f-\rho_b)\mu_0(\mathbf{x})$ and face coefficients $c = \Delta t/\rho$ directly on the MAC faces. This coupled variable-density operator departs from the single-density BDIM form of WATERLILY.JL and is therefore solved exclusively by the multigrid path.

E. Body representation: analytical vs mesh

The abstract `Body` class exposes three operations: (i) `compute_sdf` evaluates d on the (possibly staggered) grid; (ii) `compute_velocity` returns $\mathbf{v}_b$; and (iii) `update(t, it)` advances the body kinematics. Two concrete families are provided.

1) Analytical bodies: For primitives whose SDF admits a closed form we evaluate d directly on the staggered grid at each step. Built-in shapes are spheres / circles, axis-aligned boxes, capsules (useful for articulated fish tails), and NACA-foil fish spines parameterised by a cubic spline of centre-line positions [3]. Because d is evaluated analytically at every time step, arbitrary kinematics (prescribed or driven by MUJOCO) cost only a small number of elementwise GPU kernels.

2) Mesh bodies: For arbitrary triangulated geometry (STL/OBJ) the SDF is computed in a pre-processing pass and then transformed rigidly at run time. The pipeline is: closest-point queries through OPEN3D [21] ray-casting; sign determination via winding numbers; narrow-band fast marching [22] (scikit-fmm) to extend the distance field beyond the mesh; and projection onto the three MAC face grids. At simulation time the body pose (from MUJOCO or from a prescribed trajectory) rigidly transforms the cached SDF via a `pytorch-interpolation` regular-grid trilinear interpolator, so that mesh updates are a single GPU interpolation rather than a re-triangulation.

3) Composite and articulated bodies: The union of two bodies is represented through the pointwise minimum of their SDFs, $d_{\text{union}} = \min(d_1, d_2)$. A `CompositeSegmentBody` stacks the per-link SDFs of an articulated animat, while `MultiAnimatBodies` aggregates independent swimmers in the same domain. An axis-aligned bounding-box optimisation avoids evaluating each link's SDF over the full grid whenever it occupies less than $\approx 90\%$ of the domain, which is the common case for small animats in large pools.

F. Coupling with MUJOCO through FARMS

In standalone mode the `FluidSolver` simply advances Eqs. (1)–(2) together with prescribed body kinematics. In coupled mode, the solver is invoked as a FARMS `TaskExtension` (`FluidExtension`), which participates in MUJOCO's `before_step` callback. One coupled time step proceeds as follows:

1) *Pose read-out.* FARMS sensors expose link positions, orientations, and linear/angular velocities; these are copied into PYTORCH tensors.
2) *SDF assembly.* Each link updates its cached SDF (analytical re-evaluation for primitives, rigid-transform interpolation for meshes) inside a narrow AABB; the union over links is taken via $\min$.
3) *Fluid step.* The Heun predictor–corrector advances the velocity field, applying BDIM at every RK2 stage and solving the pressure Poisson equation on the current mask.
4) *Force integration.* Viscous and pressure contributions are integrated per link, plus a hydrostatic buoyancy term $F_{z,\text{buoy}} = -\rho_w(m_\ell/\rho_b)g\,\min\big((z_{\text{surf}}+h_\ell-z_\ell)/(2h_\ell),1\big)$ that smoothly switches on as a link crosses a prescribed free surface.
5) *Wrench write-back.* The six-vector per-link wrench is written to MUJOCO's `xfrc_applied` buffer and integrated by the rigid-body engine during its next step.

Because LILYTORCH and MUJOCO share a single Python process, no inter-process communication is required; the GPU

and CPU execute in alternation once per simulation step. A complementary `WaterDynamicsCallback` feeds the CFD velocity field back to MuJoCo's native drag solver for cheaper, lower-fidelity scenarios (e.g. shallow-water robots), and a `FlowViewer` extension renders vorticity isosurfaces directly inside the MuJoCo GUI and recorded video, aiding debugging.

a) Body density and buoyancy: Hydrodynamic and inertial properties of the multibody model are inherited from the MuJoCo XML (masses, inertias, collision meshes). The fluid solver additionally queries a *body density* ρ_b: when it equals the fluid density the projection reduces to the standard BDIM form; otherwise the variable-density multigrid path is taken and buoyancy is computed from the displaced volume. For mesh-based animats (e.g. the pleurodeles and salamander models shipped with the repository) the displaced volume is tabulated once from the SDF and reused; for analytical composites it is the sum of the primitive volumes.

G. Non-Newtonian rheology and sponge layer

Two additional physical models extend the solver to soft-robotic and open-domain scenarios. A Carreau fluid model [30] $\nu(\dot\gamma) = \nu_\infty + (\nu_0 - \nu_\infty)[1 + (\lambda\dot\gamma)^2]^{(n-1)/2}$ captures shear-thinning behaviour (e.g. swimming in carboxymethylcellulose gels), and a Herschel–Bulkley extension adds a yield-stress term $\tau_y/[\rho \max(\dot\gamma, \epsilon)]$ that is clamped from above by the CFL diffusion limit. A quadratic sponge layer of thickness L_s and strength $\sigma_{\max}$ damps outgoing disturbances through the post-projection update $\mathbf{u} \leftarrow \mathbf{u}/(1 + \Delta t\, \sigma)$, which emulates an infinite far-field at a fraction of the cost of perfectly-matched layers.

H. Implementation and GPU kernel fusion

The solver is written in approximately 10 k lines of Python on top of PyTorch [20]. All tensors live on a single CUDA device; multi-GPU domain decomposition is not required for the problem sizes considered here ($N_x N_y N_z \lesssim 256^3$ typical, 512^3 achievable). Hot paths — the advection–diffusion kernel, the multigrid smoother and full V-cycle, the SDF and μ_0/μ_1 evaluation, and the force integration — are expressed as *module-level pure functions* rather than methods, so that `torch.compile` with `mode="reduce-overhead"` can fuse them and capture CUDA graphs. Variants are generated per spatial dimension and per smoother choice.

YAML configuration files drive every simulation: the `solver`, `boundary_conditions`, `body`, and `output` blocks select grid, BCs, bodies, and logging; an additional `physics` block is used only when coupling with FARMS. State checkpoints are written to HDF5 and can be resumed.

III. Discussion

A. Design trade-offs

LilyTorch occupies a middle ground in the landscape of immersed-boundary solvers. Highly-optimised C++/CUDA codes such as IBAMR [15] and AMReX-based solvers offer adaptive mesh refinement and distributed-memory scaling, at the cost of a steep learning curve and a much less flexible body interface. WaterLily.jl [16] pioneered the idea of a BDIM solver that is short enough to be re-read and hackable; our work preserves that philosophy but moves the implementation to PyTorch, which grants three practical advantages. First, PyTorch's autograd and `torch.compile` infrastructure are readily available for *adjoint*-based shape and control optimisation [35], without any additional code. Second, the Python ecosystem makes it trivial to embed the solver inside MuJoCo, FARMS, or reinforcement-learning training loops, which is our primary use case. Third, every research group already familiar with PyTorch can modify the solver — for instance by swapping the advection scheme or plugging in a learned closure [36] — without cross-language FFI.

The price paid is a single-GPU ceiling: with 512^3 grids at single precision we are memory-bound on a 40 GB accelerator and cannot (yet) scale out. For the 3-D experiments reported in the companion section (free-swimming anguilliform animats, salamander paddling, single-sphere sedimentation benchmarks against Koumoutsakos & Leonard [37] and Bergmann & Iollo [38]) this ceiling is comfortable; beyond it, either domain decomposition or the existing PETSc back-end [39] is the most promising path.

B. Comparison with analytical drag models

Most robotics-oriented swimming simulators approximate hydrodynamics through link-wise drag and added-mass tensors [5], which are computationally cheap but discard wake-induced effects, inter-link shielding, and boundary corrections. By solving the full Navier–Stokes equations on a staggered MAC grid with BDIM2, LilyTorch reproduces phenomena that are structurally absent from drag-based pipelines: vortex shedding behind a pinned cylinder (validated against Koumoutsakos & Leonard [37]), thrust enhancement from body undulation, and wake–wake interaction between swimmers. The cost is roughly two to three orders of magnitude higher wall-clock time per step than a pure drag simulator, which is acceptable for scientific studies (hours to days per trajectory) but precludes, for the moment, training full reinforcement-learning policies at scale unless low-resolution proxies are used.

C. Body-representation considerations

The dual analytical/mesh body path is crucial for embodied robotics experiments. Analytical primitives keep the SDF computation tractable when an articulated animat has dozens of links whose shapes would otherwise be expensive to interpolate every step; they also sidestep the mesh-resolution / grid-resolution mismatch that can introduce high-frequency noise at the BDIM boundary. Conversely, mesh bodies are indispensable whenever the geometry comes from imaging data (fish micro-CT, salamander body scans) or has been optimised offline. Our use of Open3D ray-casting, winding numbers, and fast marching yields an SDF of quality comparable to signed-distance generators built on top of CGAL [40], but in pure Python and without C++ toolchain dependencies

— a practical advantage for a library that must be installable on a wide variety of robotics workstations. The rigid-transform interpolation performed at run time preserves single-GPU throughput: a typical STL body costs only a few extra milliseconds per step, independent of the number of triangles.

D. Coupling stability and time-step considerations

Two-way coupling with MuJoCo exposes two numerical pitfalls. *Added-mass instability* can arise when $\rho_b \approx \rho_f$ and the fluid forces dominate the body inertia [41]; for the neutrally-buoyant animats studied here we mitigate it with the variable-density projection $\rho = \rho_b + (\rho_f - \rho_b)\mu_0$, which absorbs the fluid's virtual mass into the Poisson operator rather than treating it as an explicit body force. *Temporal resolution mismatch* between the fluid (CFL-limited at $\Delta t \sim 10^{-4}$ s on a 1 cm grid) and MuJoCo (typically 10^{-3} s for contact stability) is handled by running both engines at the fluid time step; the overhead on the rigid-body side is negligible compared to the fluid cost.

Finally, the sponge layer (Section II-G) is particularly important for low-Reynolds swimmers whose pressure Poisson equation instantaneously communicates the body motion to the whole domain; a few-centimetre layer at the walls suppresses spurious recirculation without contaminating the near-field, as verified on single-sphere sedimentation benchmarks.

E. Limitations and outlook

At present LilyTorch is limited to uniform Cartesian grids and single-phase flow; free-surface extensions through a volume-of-fluid or level-set approach, and local grid refinement through an AMReX back-end, are natural next steps. Adaptive time stepping is currently manual and could be automated from the flow diagnostics already tracked by the solver (kinetic energy, enstrophy, max-CFL, max-divergence). On the robotics side, a tighter integration of the fluid forces into MuJoCo's semi-implicit integrator would address the residual added-mass instability for fully hollow bodies. Finally, because all kernels are PyTorch-native and differentiable, end-to-end adjoint optimisation of gait parameters or morphology appears feasible [35], [36] and will be the subject of follow-up work.

Acknowledgments

The author thanks the members of the BioRobotics Laboratory for discussions and for providing the experimental datasets used in the companion validation section.

References

[1] E. D. Tytell and G. V. Lauder, "The hydrodynamics of eel swimming: I. wake structure," *Journal of Experimental Biology*, vol. 207, no. 11, pp. 1825–1841, 2004.

[2] U. K. Müller, J. G. M. van den Boogaart, and J. L. van Leeuwen, "Flow patterns of larval fish: Undulatory swimming in the intermediate Flow regime," *Journal of Experimental Biology*, vol. 211, no. 2, pp. 196–205, 2008.

[3] S. Kern and P. Koumoutsakos, "Simulations of optimized anguilliform swimming," *Journal of Experimental Biology*, vol. 209, no. 24, pp. 4841–4857, 2006.

[4] A. J. Ijspeert, A. Crespi, D. Ryczko, and J.-M. Cabelguen, "From swimming to walking with a salamander robot driven by a spinal cord model," *Science*, vol. 315, no. 5817, pp. 1416–1420, 2007.

[5] M. Porez, F. Boyer, and A. J. Ijspeert, "Improved Lighthill fish swimming model for bio-inspired robots: Modeling, computational aspects and experimental comparisons," *The International Journal of Robotics Research*, vol. 33, no. 10, pp. 1322–1341, 2014.

[6] R. K. Katzschmann, J. DelPreto, R. MacCurdy, and D. Rus, "Exploration of underwater life with an acoustically controlled soft robotic fish," *Science Robotics*, vol. 3, no. 16, p. eaar3449, 2018.

[7] E. D. Tytell, C.-Y. Hsu, T. L. Williams, A. H. Cohen, and L. J. Fauci, "Interactions between internal forces, body stiffness, and fluid environment in a neuromechanical model of lamprey swimming," *Proceedings of the National Academy of Sciences*, vol. 107, no. 46, pp. 19 832–19 837, 2010.

[8] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby, "A tensorial approach to computational continuum mechanics using object-oriented techniques," *Computers in Physics*, vol. 12, no. 6, pp. 620–631, 1998.

[9] P. F. Fischer, J. W. Lottes, and S. G. Kerkemeier, "nek5000 web page," http://nek5000.mcs.anl.gov, 2008.

[10] C. S. Peskin, "The immersed boundary method," *Acta Numerica*, vol. 11, pp. 479–517, 2002.

[11] R. Mittal and G. Iaccarino, "Immersed boundary methods," *Annual Review of Fluid Mechanics*, vol. 37, pp. 239–261, 2005.

[12] G. D. Weymouth and D. K.-P. Yue, "Boundary data immersion method for Cartesian-grid simulations of fluid–body interaction problems," *Journal of Computational Physics*, vol. 230, no. 16, pp. 6233–6247, 2011.

[13] A. P. Maertens and G. D. Weymouth, "Accurate Cartesian-grid simulations of near-body flows at intermediate Reynolds numbers," *Computer Methods in Applied Mechanics and Engineering*, vol. 283, pp. 106–129, 2015.

[14] G. V. Lauder, "Fish locomotion: Recent advances and new directions," *Annual Review of Marine Science*, vol. 7, pp. 521–545, 2015.

[15] A. P. S. Bhalla, R. Bale, B. E. Griffith, and N. A. Patankar, "A unified mathematical framework and an adaptive numerical method for fluid–structure interaction with rigid, deforming, and elastic bodies," *Journal of Computational Physics*, vol. 250, pp. 446–476, 2013.

[16] G. D. Weymouth and B. Font, "WaterLily.jl: A differentiable and backend-agnostic Julia solver for incompressible viscous flow around dynamic bodies," *arXiv preprint arXiv:2407.16032*, 2024.

[17] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.

[18] J. Arreguit, S. Tata Ramalingasetty, and A. J. Ijspeert, "FARMS: Framework for animal and robot modeling and simulation," *bioRxiv*, 2023.

[19] A. J. Ijspeert, "Central pattern generators for locomotion control in animals and robots: A review," *Neural Networks*, vol. 21, no. 4, pp. 642–653, 2008.

[20] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[21] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A modern library for 3D data processing," *arXiv preprint arXiv:1801.09847*, 2018.

[22] J. A. Sethian, "A fast marching level set method for monotonically advancing fronts," *Proceedings of the National Academy of Sciences*, vol. 93, no. 4, pp. 1591–1595, 1996.

[23] M. M. Hejlesen, J. T. Rasmussen, P. Chatelain, and J. H. Walther, "A high order solver for the unbounded Poisson equation," *Journal of Computational Physics*, vol. 252, pp. 458–467, 2013.

[24] U. Trottenberg, C. W. Oosterlee, and A. Schüller, *Multigrid*. Academic Press, 2001.

[25] B. P. Leonard, "A stable and accurate convective modelling procedure based on quadratic upstream interpolation," *Computer Methods in Applied Mechanics and Engineering*, vol. 19, no. 1, pp. 59–98, 1979.

[26] M. A. Alves, P. J. Oliveira, and F. T. Pinho, "A convergent and universally bounded interpolation scheme for the treatment of advection," *International Journal for Numerical Methods in Fluids*, vol. 41, no. 1, pp. 47–75, 2003.

[27] B. van Leer, "Towards the ultimate conservative difference scheme. V. a second-order sequel to Godunov's method," *Journal of Computational Physics*, vol. 32, no. 1, pp. 101–136, 1979.

[28] J. Stam, "Stable fluids," in *Proc. SIGGRAPH*, 1999, pp. 121–128.

[29] J. Smagorinsky, "General circulation experiments with the primitive equations: I. the basic experiment," *Monthly Weather Review*, vol. 91, no. 3, pp. 99–164, 1963.

[30] R. B. Bird, R. C. Armstrong, and O. Hassager, *Dynamics of Polymeric Liquids, Volume 1: Fluid Mechanics*, 2nd ed. Wiley, 1987.
[31] A. J. Chorin, "Numerical solution of the Navier–Stokes equations," *Mathematics of Computation*, vol. 22, no. 104, pp. 745–762, 1968.
[32] R. Temam, "Sur l'approximation de la solution des équations de Navier–Stokes par la méthode des pas fractionnaires," *Archive for Rational Mechanics and Analysis*, vol. 33, pp. 377–385, 1969.
[33] F. H. Harlow and J. E. Welch, "Numerical calculation of time-dependent viscous incompressible flow of fluid with free surface," *Physics of Fluids*, vol. 8, no. 12, pp. 2182–2189, 1965.
[34] J. Makhoul, "A fast cosine transform in one and two dimensions," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 1, pp. 27–34, 1980.
[35] D. Kochkov, J. A. Smith, A. Alieva, Q. Wang, M. P. Brenner, and S. Hoyer, "Machine learning–accelerated computational fluid dynamics," *Proceedings of the National Academy of Sciences*, vol. 118, no. 21, p. e2101784118, 2021.
[36] S. L. Brunton, B. R. Noack, and P. Koumoutsakos, "Machine learning for fluid mechanics," *Annual Review of Fluid Mechanics*, vol. 52, pp. 477–508, 2020.
[37] P. Koumoutsakos and A. Leonard, "High-resolution simulations of the flow around an impulsively started cylinder using vortex methods," *Journal of Fluid Mechanics*, vol. 296, pp. 1–38, 1995.
[38] M. Bergmann and A. Iollo, "Modeling and simulation of fish-like swimming," *Journal of Computational Physics*, vol. 230, no. 2, pp. 329–348, 2011.
[39] S. Balay, S. Abhyankar, M. F. Adams *et al.*, "PETSc users manual," *Argonne National Laboratory, ANL-95/11–Rev. 3.12*, 2019.
[40] P. Alliez, S. Tayeb, and C. Wormser, "3D fast intersection and distance computation (AABB tree)," *CGAL User and Reference Manual, CGAL Editorial Board*, 2009.
[41] P. Causin, J.-F. Gerbeau, and F. Nobile, "Added-mass effect in the design of partitioned algorithms for fluid–structure problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 194, no. 42–44, pp. 4506–4527, 2005.